



Structures

UNIT-5



What ?

A structure gathers together, different atoms of information that comprise a given entity.

For example, suppose you want to store data about a **book**. You might want to store its name (a string), its price (a float) and number of pages in it (an int).



Two approaches

- ☐ Construct individual arrays, one for storing names, another for storing prices and still another for storing number of pages.
- ☐ Use a structure variable.



First approach: Array

```
#include <stdio.h>
void main( )
{
    char name[3] ;
    float price[3] ;
    int pages[3], i ;
    printf ( "\nEnter names, prices and no. of pages of 3 books\n" ) ;
    for ( i = 0 ; i <= 2 ; i++ )
        scanf ( "%c %f %d", &name[i], &price[i], &pages[i] );
    printf ( "\nAnd this is what you entered\n" ) ;
    for ( i = 0 ; i <= 2 ; i++ )
        printf ( "%c %f %d\n", name[i], price[i], pages[i] );
}
```



Second Approach: **Structures**

```
main( )
{
    struct book
    {
        char name ;
        float price ;
        int pages ;
    };
    struct book b1, b2, b3 ;

    printf ( "\nEnter names, prices & no. of pages of 3 books\n" ) ;
    scanf ( "%c %f %d", &b1.name, &b1.price, &b1.pages ) ;
    scanf ( "%c %f %d", &b2.name, &b2.price, &b2.pages ) ;
    scanf ( "%c %f %d", &b3.name, &b3.price, &b3.pages ) ;

    printf ( "\nAnd this is what you entered" ) ;
    printf ( "\n%c %f %d", b1.name, b1.price, b1.pages ) ;
    printf ( "\n%c %f %d", b2.name, b2.price, b2.pages ) ;
    printf ( "\n%c %f %d", b3.name, b3.price, b3.pages ) ;
}
```



Declaring a Structure

```
struct <structure name>
{
    structure element 1 ;
    structure element 2 ;
    structure element 3 ;
    .....
    .....
};
```

```
struct book
{
    char name ;
    float price ;
    int pages ;
};
```



Variables declarations

Once the new structure data type has been defined one or more variables can be declared to be of that type. For example, the variables **b1**, **b2**, **b3** can be declared to be of the **type struct book**, as,

```
struct book b1, b2, b3 ;
```



All Declarations are the same!!

```
struct book
{
    char name ;
    float price ;
    int pages ;
};
struct book b1, b2, b3 ;
```

```
struct book
{
    char name ;
    float price ;
    int pages ;
} b1, b2, b3 ;
```

```
struct
{
    char name ;
    float price ;
    int pages ;
} b1, b2, b3 ;
```



Structure variables can also be initialized where they are declared.

```
struct book
{
    char name[10];
    float price;
    int pages;
};
struct book b1 = { "Basic", 130.00, 550 };
struct book b2 = { "Physics", 150.80, 800 };
```



Note the following points while declaring a structure type:

- ❑ A semicolon must follow the closing brace in the structure type declaration.
- ❑ It is important to understand that a structure type declaration does not tell the compiler to reserve any space in memory. All a structure declaration does is, it define the **‘form’ of the structure**.
- ❑ Usually structure type declaration appears at the top of the source code file, before any variables or functions are defined. In very large programs they are usually put in a separate header file, and the file is included (using the preprocessor directive `#include`) in whichever program we want to use this structure type.



Accessing Structure Elements

Structures use a different scheme. They use a **dot (.) operator**. So to refer to pages of the structure defined in our sample program we have to use,

b1.pages

b1.price

Note that before the dot there must always be a structure variable and after the dot there must always be a structure element.



Program 1:

Create a structure to specify data on students given below:

Roll number, Name, Department, Course, Year of joining

Assume that there are not more than 450 students in the collage.

- (a) Write a function to print names of all students who joined in a particular year.
- (b) Write a function to print the data of a student whose roll number is given.



Solution

<https://docs.google.com/document/d/1C-OThISbKqFbtkGO1hHStO9Y2uPZed3tNr57cjMtxxs/edit?usp=sharing>



How Structure Elements are Stored

Whatever be the elements of a structure, they are always stored in **contiguous memory locations**.

C Structure Padding and Packing

Technically, the size of the structure in C should be the sum of the sizes of its members. But it may not be true for most cases. The reason for this is Structure Padding.

Structure padding is the concept of adding multiple empty bytes in the structure to naturally align the data members in the memory. It is done to minimize the CPU read cycles to retrieve different data members in the structure.

There are some situations where we need to pack the structure tightly by removing the empty bytes. In such cases, we use **Structure Packing**. C language provides two ways for structure packing:

1. Using `#pragma pack(1)`

2. Using `__attribute__((packed))`

<https://www.geeksforgeeks.org/structures-c/> =====> Read



Usage

```
/* Memory map of structure elements */
#include <stdio.h>
void main( )
{
    struct book
    {
        char name ;
        float price ;
        int pages ;
    };
    struct book b1 = { 'B', 130.00, 550 } ;
    printf("Size of structure is %d",sizeof(b1));
    printf ( "\nAddress of name = %u", &b1.name );
    printf ( "\nAddress of price = %u", &b1.price ) ;
    printf ( "\nAddress of pages = %u", &b1.pages ) ;
}
```

Size of structure is 12

Address of name = 340697764

Address of price = 340697768

Address of pages = 340697772



Array of Structures

```
// Array of structures
#include <stdio.h>
void main()
{
    struct book
    {
        char name[25];
        int price;
        int pages;
    };
    struct book b[5];
    int i;
    for(i=0;i<2;i++)
    {
        printf("Enter the name, price and pages: \n");
        scanf("%s %d %d",b[i].name,&b[i].price,&b[i].pages);
    }
    for(i=0;i<2;i++)
        printf("%s %d %d\n",b[i].name,b[i].price,b[i].pages);
}
```



Additional Features of Structures

1. The values of a structure variable can be assigned to another structure variable of the same type using the assignment operator.

```
#include <stdio.h>
#include <string.h>
void main( )
{
    struct employee
    {
        char name[10] ;
        int age ;
        float salary ;
    };
    struct employee e1 = { "Sanjay", 30, 5500.50 } ;
    struct employee e2, e3 ;
    /* piece-meal copying */
    strcpy ( e2.name, e1.name ) ;
    e2.age = e1.age ;
    e2.salary = e1.salary ;
    /* copying all elements at one go */
    e3 = e2 ;
    printf ( "\n%s %d %f", e1.name, e1.age, e1.salary ) ;
    printf ( "\n%s %d %f", e2.name, e2.age, e2.salary ) ;
    printf ( "\n%s %d %f", e3.name, e3.age, e3.salary ) ;
}
```



2) One structure can be nested within another structure.

```
#include <stdio.h>
void main( )
{
    struct address
    {
        char phone[15] ;
        char city[25] ;
        int pin ;
    };
    struct emp
    {
        char name[25] ;
        struct address a ;
    };
    struct emp e = { "jeru", "531046", "nagpur", 10 };
    printf ( "\nname = %s phone = %s", e.name, e.a.phone ) ;
    printf ( "\ncity = %s pin = %d", e.a.city, e.a.pin ) ;
}
```

```
name = jeru phone = 531046
city = nagpur pin = 10
```



3) Like an ordinary variable, a structure variable can also be passed to a function. We may either pass **individual structure elements** or the **entire structure variable** at one go.

```
/* Passing individual structure elements */
#include <stdio.h>
void display(char *, char *, int);
void main( )
{
    struct book
    {
        char name[25] ;
        char author[25] ;
        int callno ;
    };
    struct book b1 = { "Let us C", "YPK", 101 };
    display ( b1.name, b1.author, b1.callno );
}
void display ( char *s, char *t, int n )
{
    printf ( "\n%s %s %d", s, t, n ) ;
}
```

Let us C YPK 101



Passing Individual structure
elements



```
#include <stdio.h>
struct book
{
    char name[25] ;
    char author[25] ;
    int callno ;
};
void display(struct book);
void main( )
{
    struct book b1 = { "Let us C", "YPK", 101 } ;
    display ( b1 ) ;
}
void display ( struct book b )
{
    printf ( "\n%s %s %d", b.name, b.author, b.callno ) ;
}
```

Let us C YPK 101



Entire structure variable



4) The way we can have a pointer pointing to an int, or a pointer pointing to a char, similarly we can have a pointer pointing to a struct. Such pointers are known as 'structure pointers'.

```
#include <stdio.h>
void main( )
{
    struct book
    {
        char name[25] ;
        char author[25] ;
        int callno ;
    } ;
    struct book b1 = { "Let us C", "YPK", 101 } ;
    struct book *ptr ;
    ptr = &b1 ;
    printf ( "\n%s %s %d", b1.name, b1.author, b1.callno ) ;
    printf ( "\n%s %s %d", ptr->name, ptr->author, ptr->callno ) ;
}
```

```
Let us C YPK 101
Let us C YPK 101
```



5) In such cases C provides an operator `->`, called an **arrow operator** to refer to the structure elements. Remember that on the left-hand side of the `'` structure operator, there must always be a structure variable, whereas on the **left-hand side of the `'->` operator** there must always be a pointer to a structure.

b1.name	b1.author	b1.callno
Let Us C	YPK	101
65472	65497	65522

ptr
65472
65524



Can we not pass the address of a structure variable to a function? **We can.** The following program demonstrates this.

```
/* Passing address of a structure variable */
#include <stdio.h>
struct book
{
    char name[25] ;
    char author[25] ;
    int callno ;
};
void display ( struct book * );
void main( )
{
    struct book b1 = { "Let us C", "YPK", 101 } ;
    display ( &b1 ) ;
}
void display ( struct book *b )
{
    printf ( "\n%s %s %d", b->name, b->author, b->callno ) ;
}
```

Let us C YPK 101



The **#pragma pack** directive offers a way to fulfill this requirement. This directive specifies packing alignment for structure members. The pragma takes effect at the first structure declaration after the pragma is seen. Turbo C/C++ compiler doesn't support this feature, VC++ compiler does. The following code shows how to use this directive.

```
#include <stdio.h>
#pragma pack(1)
struct emp
{
    int a;
    char ch;
    float s;
};
#pragma pack( )
struct emp e ;
printf ("%u %u %u", &e.a, &e.ch, &e.s);
```

Here, **#pragma pack (1)** lets each structure element to begin on a 1-byte boundary



Uses of Structures

- (a) Changing the size of the cursor
- (b) Clearing the contents of the screen
- (c) Placing the cursor at an appropriate position on screen
- (d) Drawing any graphics shape on the screen
- (e) Receiving a key from the keyboard
- (f) Checking the memory size of the computer
- (g) Finding out the list of equipment attached to the computer
- (h) Formatting a floppy
- (i) Hiding a file from the directory
- (j) Displaying the directory of a disk
- (k) Sending the output to printer
- (l) Interacting with the mouse



Practice Programs

- (a) Create a structure to specify data on students given below:

Roll number, Name, Department, Course, Year of joining

Assume that there are not more than 450 students in the collage.

- (a) Write a function to print names of all students who joined in a particular year.
- (b) Write a function to print the data of a student whose roll number is given.

Practice Programs

- (b) Create a structure to specify data of customers in a bank. The data to be stored is: Account number, Name, Balance in account. Assume maximum of 200 customers in the bank.
 - (a) Write a function to print the Account number and name of each customer with balance below Rs. 100.
 - (b) If a customer request for withdrawal or deposit, it is given in the form:

Acct. no, amount, code (1 for deposit, 0 for withdrawal)

Write a program to give a message, “The balance is insufficient for the specified withdrawal”.



Practice Programs

Write a menu driven program that depicts the working of a library. The menu options should be:

1. Add book information
2. Display book information
3. List all books of given author
4. List the title of specified book
5. List the count of books in the library
6. List the books in the order of accession number
7. Exit

Create a structure called **library** to hold accession number, title of the book, author name, price of the book, and flag indicating whether book is issued or not.



Practice Programs

Write a program that compares two given dates. To store date use structure say **date** that contains three members namely date, month and year. If the dates are equal then display message as "Equal" otherwise "Unequal".



Practice Programs

There is a structure called **employee** that holds information like employee code, name, date of joining. Write a program to create an array of the structure and enter some data into it. Then ask the user to enter current date. Display the names of those employees whose tenure is 3 or more than 3 years according to the given current date.